Deeper Networks, Built from Blocks

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Explain what a receptive field is, and why a stack of small convolutions can recognize something much larger than any one kernel
- Build a deep network from a repeated block (the VGG block), and say what changes between blocks: spatial size halves, channels grow
- Compare a stack of 3×3 convolutions with one larger kernel, in receptive field and in parameter count
- Describe what a 1×1 convolution computes, and how global average pooling replaces a large linear layer at the end of the network
- Explain the degradation problem, and why a residual block ($\mathbf{y} = \mathbf{x} + f(\mathbf{x})$) makes a very deep network trainable
- Write a residual block as an `nn.Module`, including the shortcut when the shapes change
- Say what the layers of a trained CNN represent, and why the output of the last block is reusable

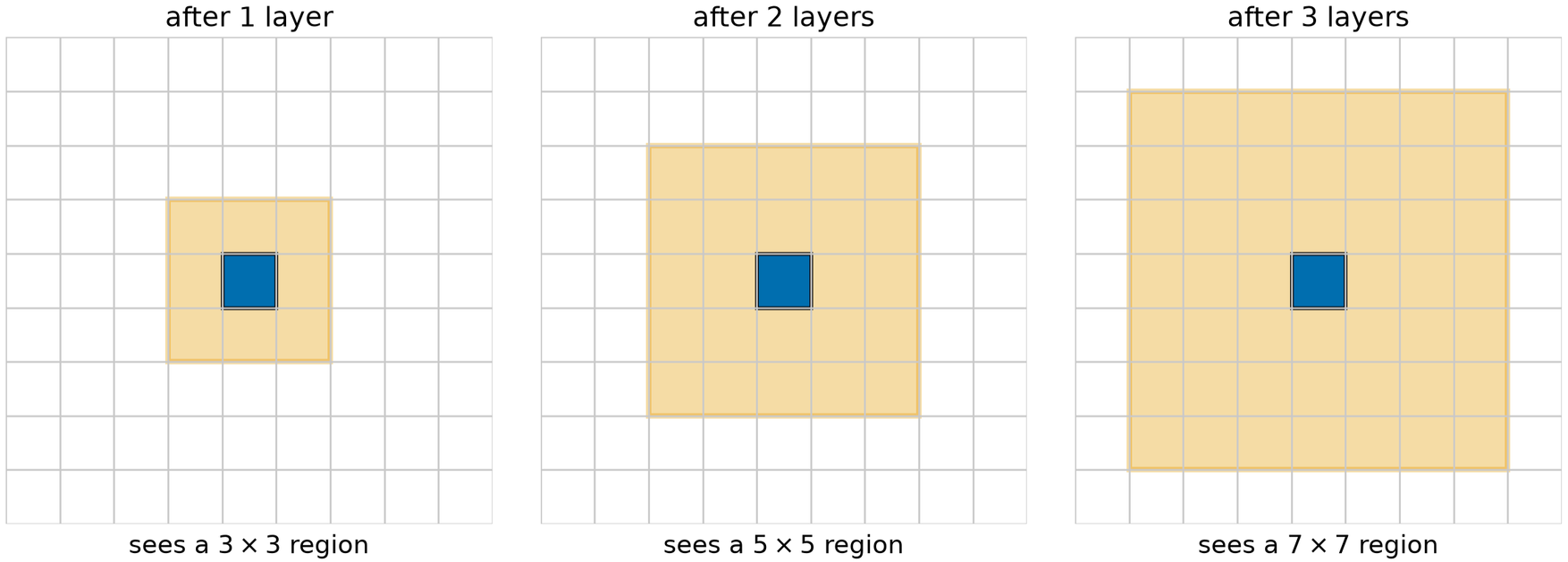
Why go deeper

What can a 3×3 filter possibly see?

A small filter, a big picture

- The CNN we built had two convolutional layers and read 28×28 grey digits
- A photograph is 224×224 and in colour, and the thing we want to recognize (a dog, a face) can fill most of it
- But every filter we have looks at a 3×3 **patch**
- How can a model made of tiny windows recognize something large?

What one output value can see



- The blue cell is one output value. The orange square is the part of the **original image** that can affect it
- Layer 2 looks at a 3×3 window of layer 1's outputs, and each of those looked at its own 3×3 window of the image

The receptive field, with pooling

Layer	Feature map	Region of the image one value sees
conv 3×3	$C \times 224 \times 224$	3×3
conv 3×3	$C \times 224 \times 224$	5×5
max pool 2×2	$C \times 112 \times 112$	6×6
conv 3×3	$C \times 112 \times 112$	10×10
conv 3×3	$C \times 112 \times 112$	14×14
max pool 2×2	$C \times 56 \times 56$	16×16
conv 3×3	$C \times 56 \times 56$	24×24
conv 3×3	$C \times 56 \times 56$	32×32

- Each convolution adds a little. Each **pooling layer doubles how much every later layer adds**, because one cell now stands for a 2×2 block
- Eight small layers later, one value sees a 32×32 region

From edges to objects

- Early layers see a tiny region, so all they can detect is **simple patterns**: edges, corners, color changes
- Middle layers combine those into **textures and parts**: a grid, a curve, an eye
- Late layers combine parts into **whole objects**: a face, a wheel, a dog
- Nobody tells the network to organize itself this way. The only requirement is that each layer reads the one before it, and training keeps whichever filters help the final answer

Same recipe, bigger

Same as our CNN

- Convolution, ReLU, max pooling, repeated
- A linear classifier on the flattened features at the end
- Cross-entropy loss, trained by gradient descent

What the 2012 ImageNet winner (AlexNet) added

- 5 convolutional layers and 3 linear layers, for 224×224 colour photos and 1,000 classes
- ReLU, instead of sigmoid, so a deeper stack still trains
- Dropout in the linear layers, and data augmentation
- GPUs, and 1.2 million training images

Takeaway

Stack many small convolutions: each layer looks at patterns of the layer before, so deeper layers see larger structures built out of smaller ones. The ingredients from last time (ReLU, dropout, augmentation, more data) are what made it possible to train.

Blocks: VGG

How do you design a network with 11 layers, or 100?

Too many choices

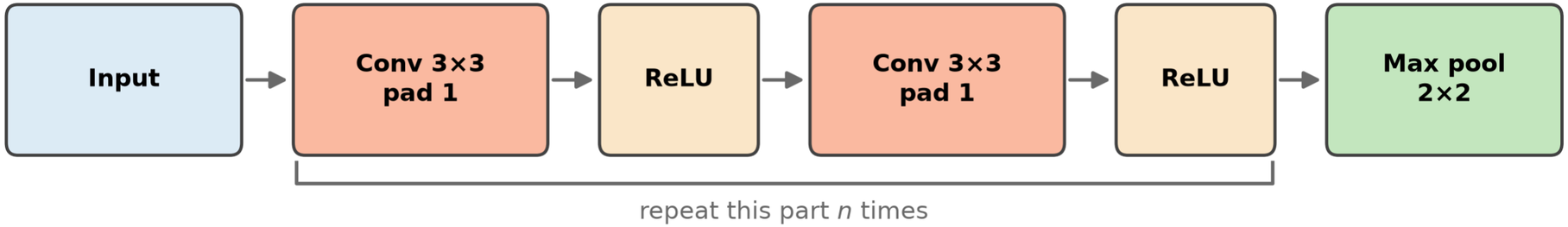
- Every layer needs a kernel size, a channel count, and a stride, and whether it pools
- With 10 layers that is about 40 decisions, and 100 layers is out of the question
- Trying them one at a time is hopeless, and they interact
- And even if we had the time, how would we **describe** the result to someone else?

> **Tip**
>
> These would all be **hyperparameters** that we'd be changing.
>
> Hyperparameters are parameters that aren't trained during SGD, but instead set prior to the training run.

The Big Idea

Design one small repeating block, then stack it. The network is described by how many blocks, and how wide each one is.

The VGG block



Why 3×3? Same view, fewer weights

- To see a 5×5 region we have two options: **one** 5×5 convolution, or **two** 3×3 convolutions in a row
- Both give a value that depends on the same 5×5 patch of the image, as in the receptive-field picture
- So the question is which option needs fewer weights to learn

Why 3×3? Counting the weights

- Say a layer takes c channels in and gives c channels out. Every (input channel, output channel) pair has its own kernel, so there are $c \cdot c = c^2$ kernels
- Each kernel has one weight per position: $3 \cdot 3 = 9$ for a 3×3 kernel, $5 \cdot 5 = 25$ for a 5×5 kernel (biases ignored)

$$\underbrace{2 \cdot (3 \cdot 3) \cdot c^2}_{\text{two } 3\times3 \text{ layers}} = 18c^2 \qquad \text{vs.} \qquad \underbrace{(5 \cdot 5) \cdot c^2}_{\text{one } 5\times5 \text{ layer}} = 25c^2$$

- The stack is cheaper, and it has a ReLU between its layers, so it can compute things a single layer cannot

Why 3×3? With real numbers

Same region seen	Stack of 3×3	One larger kernel	Saved
5×5	2 layers: 73,728	102,400	28%
7×7	3 layers: 110,592	200,704	45%

- With $c = 64$ channels. The gap widens as the region grows: **small kernels, stacked, are cheaper and more expressive**

The block in PyTorch

```
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)
```

- A function that **returns a network piece**: build it once, call it as many times as we like
- The whole network is then a list of `(num_convs, out_channels)` pairs

A network from a list

```
def vgg(conv_arch, in_channels=3, num_classes=1000):
    blocks = []
    for num_convs, out_channels in conv_arch:
        blocks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels
    return nn.Sequential(
        *blocks,
        nn.Flatten(),
        nn.LazyLinear(4096), nn.ReLU(), nn.Dropout(0.5),
        nn.LazyLinear(4096), nn.ReLU(), nn.Dropout(0.5),
        nn.LazyLinear(num_classes)
    )

conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))     # VGG-11
```

- Five blocks (8 convolutions) then three linear layers: **11 weight layers**
- Changing the network means changing the list, not the code

Tracing the shapes

After	Output shape
input	`(1, 3, 224, 224)`
block 1	`(1, 16, 112, 112)`
block 2	`(1, 32, 56, 56)`
block 3	`(1, 64, 28, 28)`
block 4	`(1, 128, 14, 14)`
block 5	`(1, 128, 7, 7)`

- Shown with channels divided by 4 so it runs instantly. In full VGG-11 the channels are $64, 128, 256, 512, 512$

Halve the size, grow the channels

- Every block **halves** the height and width ($224 \to 112 \to 56 \to 28 \to 14 \to 7$)
- Every block (up to a cap) **grows** the number of channels
- Early on: many positions, few kinds of pattern. Late: few positions, many kinds of pattern
- The picture shrinks and the **representation gets richer**: “where exactly” is traded for “what is here”

Takeaway

A block is the unit of design. Stacking blocks that halve the size and grow the channels is how we get depth without choosing every layer by hand.

Looking inside the head: 1×1 and averaging

Where do the parameters actually live?

The parameters are in the wrong place

Model	Total parameters	In the final linear layers	Share
AlexNet	61,100,840	58,631,144	96%
VGG-11	132,863,336	123,642,856	93%
ResNet-18	11,689,512	513,000	4%

- In AlexNet and VGG, almost everything we pay for is in the **last few linear layers**, not the convolutions
- VGG's first linear layer alone: $512 \times 7 \times 7 = 25{,}088$ features in, 4096 out, so $102{,}760{,}448$ weights
- ResNet-18 manages with a tiny head. How?

The Big Idea

The convolutional layers already did the hard work. Instead of a giant linear layer to read their output, mix the channels at each position, then average each channel over all positions.

The 1×1 convolution

- A kernel of size 1×1 looks at **one position** and **all the channels** at that position
- It is a linear layer applied **independently at every pixel**, mixing the channels and ignoring neighbours
- Shape: $c_{in} \times H \times W \to c_{out} \times H \times W$. The height and width do not change
- Parameters: $c_{in} \cdot c_{out}$ (plus c_{out} biases), no matter how big the picture is

A 1×1 convolution, by hand

- One position with $c_{in} = 3$ channels: $(2, 1, 3)$. Two output channels, one weight vector each:

$$o_1 = 2 \cdot 1 + 1 \cdot 0 + 3 \cdot (-1) = -1 \qquad o_2 = 2 \cdot 0.5 + 1 \cdot 0.5 + 3 \cdot 0.5$$

- Weights $(1, 0, -1)$ for channel 1 and $(0.5, 0.5, 0.5)$ for channel 2
- The same six weights are applied at **every** position of the feature map. That is weight sharing, again, only across channels instead of across space

Global average pooling

- Take each channel's $H \times W$ feature map and replace it with its **average**. One number per channel
- Example, two channels of a 2×2 feature map:

$$\begin{matrix} 1 & 3 \\ 5 & 7 \end{matrix} \rightarrow \frac{1+3+5+7}{4} = 4 \qquad \begin{matrix} 0 & 2 \\ 2 & 0 \end{matrix} \rightarrow \frac{0+2+2+0}{4} = 1$$

- A $2 \times 2 \times 2$ block of numbers becomes just $(4, 1)$: one number answering "**how much of this pattern is there, anywhere?**"
- No parameters at all, and it works for any image size

A new head

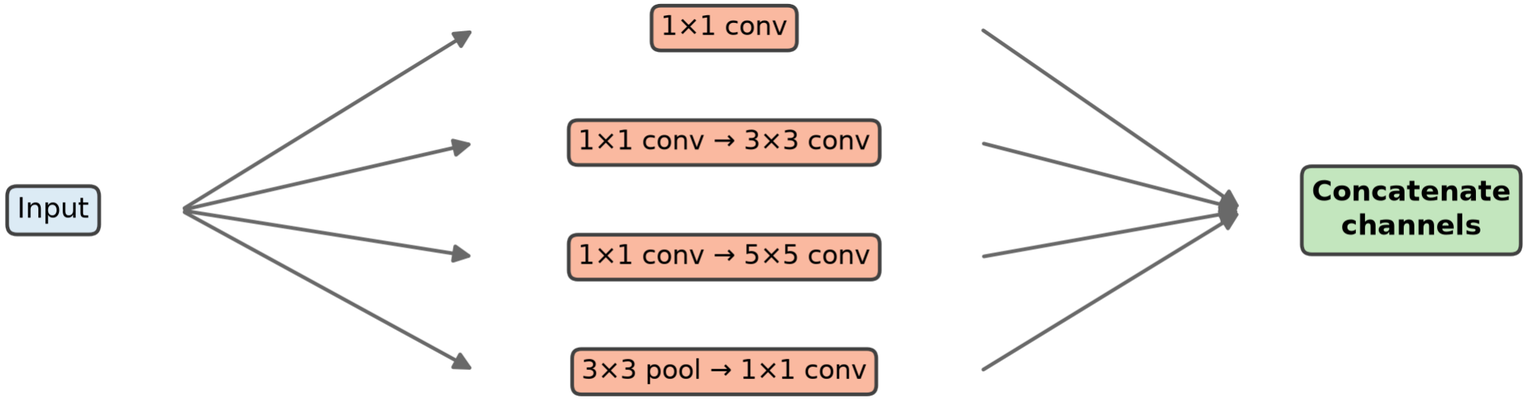
Flatten, then big linear layers

- $512 \times 7 \times 7 = 25{,}088$ numbers
- Linear to 4096: ≈ 102.8M weights
- Tied to exactly this image size

Global average pool, then one linear layer

- 512 numbers, one per channel
- Linear to 1000 classes: $512 \cdot 1000 + 1000 = 513{,}000$ weights
- Works for any image size

Not every block is a straight line



- GoogLeNet's **Inception block**: several filter sizes **in parallel** on the same input, then stack the results along the channel axis. The network can pick the sizes it wants
- The 1×1 convolutions in front keep each branch cheap by shrinking the channels first
- A block can have more than one path through it. Hold that thought

Takeaway

A 1×1 convolution mixes channels at each position, and global average pooling collapses positions. Together they replace a huge linear layer with a few hundred thousand weights.

Residual networks

Why can’t we just keep stacking?

More layers should never hurt

- A deeper network can do everything a shallower one can: let the extra layers pass their input through unchanged, and we have the shallow network back
- So adding layers should never make the best possible result worse
- In practice, past a certain depth, **it does**
- Add layers to a plain stack and the model gets worse. Where does the problem come from?

The degradation problem

- In the original ResNet paper (He et al., 2015), a 56-layer plain network did **worse than a 20-layer one** on CIFAR-10
- Not just on the validation set. On the **training set** too
- Batch norm and ReLU were already in use, so this was not the vanishing-gradient problem from before, at least not in its simplest form
- The extra layers could, in principle, have copied the 20-layer solution. The optimizer **failed to find it**

Not overfitting

Overfitting: training error is low, validation error is high.
Degradation: **training** error is high.

- A bigger model that fits the training data *worse* is not memorizing too much. It is **not being trained well enough**
- From the curve shapes last time: this is a “both curves are bad” problem, not a gap problem
- So the fix is not more data or dropout. It has to change **how the layers are connected**

The Big Idea

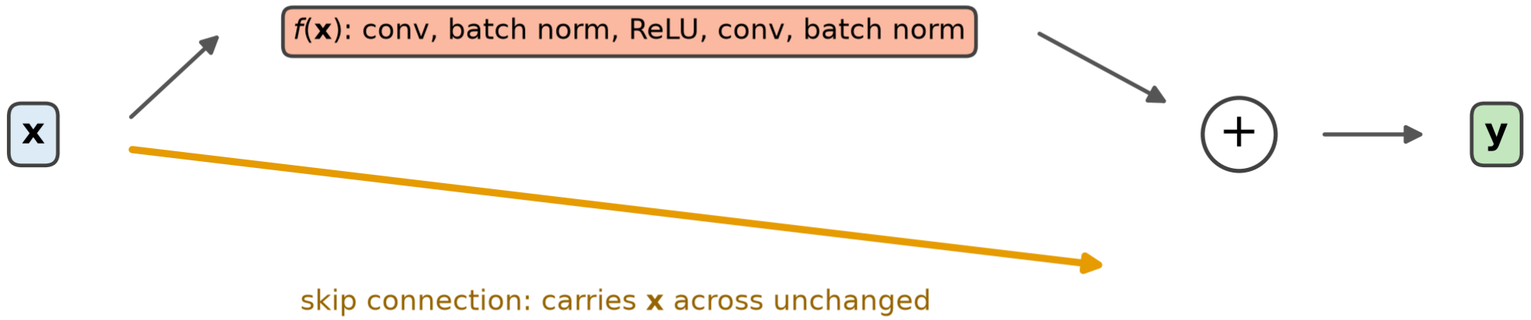
Do not make each block learn the whole output. Let it learn only the change to its input, and add the input back.

The residual block

$$\mathbf{y} = \mathbf{x} + f(\mathbf{x})$$

- $\mathbf{x}$ is the block's input, $\mathbf{y}$ its output
- f is a few layers of convolution, batch norm and ReLU, the same kind of thing we have been stacking
- The input is **added** to the output of f, so the two must have the same shape
- The path that carries $\mathbf{x}$ straight across is the **skip connection** (or shortcut)

Breakdown: the skip connection



- The skip path has **no parameters and no computation**: it just copies
- Whatever else the block does, the input is still there at the output

Breakdown: the residual function

- f only has to learn **what to add to $\mathbf{x}$**, not to rebuild $\mathbf{x}$ from scratch
- If the best thing for this block is to do nothing, f only has to produce zeros. For a plain stack, the layers would have to learn the identity exactly
- So adding a block can never make the network *start* worse: with $f \approx 0$, it is the same network as before

Same shape, different name

Gradient descent

$$\mathbf{w}_{new} = \mathbf{w} + \Delta\mathbf{w}$$

- Keep the old value
- Add a small correction

A residual block

$$\mathbf{y} = \mathbf{x} + f(\mathbf{x})$$

- Keep the old value
- Add a small correction

- The network is refined by a sequence of small corrections, instead of rewritten at every layer

One value through a block

- Say one entry of $\mathbf{x}$ is 3.0 and the block's layers compute $f = 0.5$
- Plain layer: the output is whatever the layers say, 0.5. The 3.0 is gone
- Residual block: $y = 3.0 + 0.5 = 3.5$. The 3.0 is kept, with a nudge
- If training pushes the layers' weights to zero, $f = 0$, so $y = 3.0$: **the identity, for free**

Why gradients survive

- Differentiate $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$ with respect to $\mathbf{x}$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = 1 + \frac{\partial f}{\partial \mathbf{x}}$$

- The 1 comes from the skip. Backprop multiplies one of these per block, going backwards
- Suppose, for illustration, every layer's contribution is a factor of 0.1 (a made-up number, not measured). Over 8 layers:

Plain	Residual
$0.1^8 = 10^{-8}$	$(1 + 0.1)^8 \approx 2.1$

When the shapes do not match

- The sum $\mathbf{x} + f(\mathbf{x})$ needs both to have the same shape
- When a block changes the number of channels or halves the size (stride 2), the skip path cannot copy $\mathbf{x}$ as it is
- Fix: put a 1×1 **convolution** (with the same stride) on the skip path. It mixes the channels to the new count and skips positions to match
- Blocks that keep the shape use the plain copy. Only the blocks that change it pay for the extra convolution

The block in PyTorch

```
 1 class Residual(nn.Module):
 2     def __init__(self, in_channels, out_channels, stride=1):
 3         super().__init__()
 4         self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=stride, padding=1)
 5         self.bn1 = nn.BatchNorm2d(out_channels)
 6         self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
 7         self.bn2 = nn.BatchNorm2d(out_channels)
 8         self.skip = None
 9         if in_channels != out_channels or stride != 1:
10             self.skip = nn.Conv2d(in_channels, out_channels, 1, stride=stride)
11 
12     def forward(self, x):
13         out = F.relu(self.bn1(self.conv1(x)))
14         out = self.bn2(self.conv2(out))
15         shortcut = x if self.skip is None else self.skip(x)
16         return F.relu(out + shortcut)
```

- `forward` is the formula: `out` is $f(\mathbf{x})$, `shortcut` is $\mathbf{x}$, and the block returns their sum (then a ReLU)

Block	Input	Output	Skip path	Parameters
`Residual(64, 64)`	`(1, 64, 56, 56)`	`(1, 64, 56, 56)`	copy	74,112
`Residual(64, 128, stride=2)`	`(1, 64, 56, 56)`	`(1, 128, 28, 28)`	1×1 conv	230,272

ResNet-18

Part	What it is	Output shape
Stem	7×7 conv, stride 2, then 3×3 max pool, stride 2	$64 \times 56 \times 56$
Stage 1	2 residual blocks, 64 channels	$64 \times 56 \times 56$
Stage 2	2 residual blocks, 128 channels, the first at stride 2	$128 \times 28 \times 28$
Stage 3	2 residual blocks, 256 channels, the first at stride 2	$256 \times 14 \times 14$

Part	What it is	Output shape
Stage 4	2 residual blocks, 512 channels, the first at stride 2	$512 \times 7 \times 7$
Head	Global average pool, then one linear layer	1000

- Same pattern as VGG: halve the size, double the channels. 18 weight layers: 1 in the stem, 16 in the blocks, 1 in the head
- About 11.7M parameters, roughly one-eleventh of VGG-11's, and it trains better

Takeaway

A residual block adds its input back to its output, so every block starts as “do nothing” and learns a correction. That is what makes networks with dozens, and even hundreds, of layers trainable.

What the layers learned

Reusing the representation

The head is the same linear classifier

- A ResNet is a stack of blocks that turns a photograph into **512 numbers** (after the global average pool)
- The final layer, `Linear(512, 1000)`, is the same linear classifier we have used since the first classification lecture
- All the work of making classes **linearly separable** happened in the blocks before it
- Nothing about those 512 numbers is specific to the 1,000 classes the network was trained on. They describe how much of each learned pattern is present

A trained network is a feature extractor

- Take a network trained on a huge dataset, keep every block, and throw away only the last linear layer
- What is left maps **any** image to a vector of features that describes it
- Train a new, small linear classifier on those features for **our own** problem, with our own classes and far less data
- That is **transfer learning**, which is where we go next

Takeaway

Deep networks learn representations in layers, from edges to objects. Those representations are not only the means to one classification: they are reusable.

Summary

Recap: what each design contributes

Idea	What it does	Why it matters
Stacked small kernels	Receptive field grows layer by layer	Large structures from small filters, fewer parameters
The block (VGG)	One repeated unit; halve the size, grow the channels	Describe a deep net in a list
1×1 convolution	Mix channels at each position	Reshape channel counts cheaply
Global average pooling	One number per channel	Replaces a huge linear head

Recap: key ideas

- Each layer reads the one before it, so **deeper layers see larger structures**: edges, then parts, then objects
- Design a **block** and repeat it, rather than choosing every layer
- Most parameters can hide in the linear head. 1×1 **convolutions and global average pooling** remove it
- A plain stack gets **harder to train** with depth, even on the training set. That is not overfitting
- **Residual blocks** keep the input and learn a correction, so the gradient always has a direct path

Where residuals show up next

- The residual connection is not just a CNN trick. It appears again, almost unchanged, in the **Transformer** blocks at the heart of language models
- Whenever we meet a very deep stack, expect an $\mathbf{x} + f(\mathbf{x})$ inside it
- Next time: using a trained network's representations for a new problem, and looking at where such a model goes wrong